Boot Controller Architecture

SONIC-MCU

Hardware-Synchronized DMA Pipeline | SPEC-BOOTFSM-v16.0

Introduction

The Boot Controller is a dedicated hardware state machine responsible for initializing the system memory post-reset. To eliminate the need for a complex software-based Boot ROM executing directly from slow flash, this hardware module seizes the system buses upon reset, orchestrates the SPI peripheral, and funnels executable code and initialized data into the fast Instruction SRAM (I-SRAM) and Data SRAM (D-SRAM).

The Boot Controller acts as an intermediary possessing **two distinct bus interfaces**:

1. **APB Interface:** Used to issue configuration writes to the SPI peripheral and read incoming firmware data from the SPI `RX_FIFO`.
2. **AXI-Lite Interface:** Connected to the main system crossbar to perform high-speed writes of the fetched instructions and data into the I-SRAM and D-SRAM.

Reset Latching

The external reset provided to the SONIC-MCU is a **single-cycle pulse**. Because this pulse is fleeting, the Boot Controller FSM is explicitly designed to **latch onto it**. Upon detecting this single-cycle reset, the FSM instantly transitions out of `IDLE` and asserts a `core_halt` signal. This freezes the RISC-V CPU pipeline safely before it can attempt to fetch invalid instructions from uninitialized memory.

SPI Flash and Bus Architecture

The Boot Controller leverages the existing SPI peripheral to interface with the external SPI Flash. Instead of employing fragmented software polling, the architecture utilizes a **hardware-synchronized, independent DMA pipeline**. The SPI Master IP and the Boot Controller FSM operate as two fully decoupled hardware blocks operating concurrently.

SPI Master Target Support and Chip Select Bitmask Logic

The `apb_spi_master` IP provides four distinct Chip Select lines (`spi_csn0` through `spi_csn3`) to support multiple peripherals on the same physical SPI bus.

A common misconception is that the hardware strictly enforces a multiplexed, 1:1 transaction model by treating the `csreg` field as a simple numerical index (e.g., `csreg == 0` selects `CSN0`). In reality, the `csreg` field operates as a **bitmask**. Each bit in the register independently controls its corresponding `CSN` output. This behavior is explicitly defined in the RTL:

Listing 1: Chip Select Bitmask Logic (spi_master_controller.sv)

```
assign spi_csn0 = ~spi_csreg[0] | spi_cs;
assign spi_csn1 = ~spi_csreg[1] | spi_cs;
assign spi_csn2 = ~spi_csreg[2] | spi_cs;
assign spi_csn3 = ~spi_csreg[3] | spi_cs;
```

Breaking this down: the internal spi_cs state is 1 when idle, and 0 during an active transaction. The spi_csreg array is a 4-bit configuration field located within the REG_STATUS register (bits 11:8). The Boot Controller FSM programs this field via the APB bus to select the active slave device(s).

- **When idle (spi_cs=1):** The logical OR forces the result to 1 regardless of spi_csreg — all lines are safely deasserted.
- **When active (spi_cs=0):** The result evaluates strictly to ~spi_csreg[x]. Thus, CSNx is asserted (driven LOW) if and only if spi_csreg[x] = 1.

Because it is a bitmask, configuring csreg = 4'b0011 would logically assert *both* CSN0 and CSN1 simultaneously.

To be precise about what "independent" means here: the Boot Controller **can** independently choose *which* slave (or slaves) are selected on any given transaction — csreg = 4'b0001 wakes only Slave 0, 4'b0010 wakes only Slave 1, 4'b1000 wakes only Slave 3, and so on. Chip-select targeting is fully flexible and can point at any single slave in a strict 1-to-1 manner.

What is **not** independent is the data path. The apb_spi_master IP shares a single common data bus across all four slaves: spi_sdo0 is the only transmit line and spi_sdi1 is the only receive line in standard mode, hardwired in the RTL—there is no register or configuration field that redirects data to or from any other SDO/SDI pin. Regardless of which CSN is asserted, every transaction uses the exact same physical wires: data is always sent out on sdo0 and always received in on sdi1. You cannot send different data to Slave 2 than to Slave 3, nor can you receive from one slave while transmitting different data intended for another—the payload is always the same broadcast over the shared bus, only the destination (via CSN) changes.

This is why asserting multiple CSN lines at once (csreg = 4'b0011) is a true broadcast rather than two independent transactions: the identical data on sdo0 reaches both Slave 0 and Slave 1 at the same time, and if both respond, they collide on the shared sdi1 return path. In practice, the Boot Controller always operates in a strict sequential model: assert exactly one CSN, complete that full transaction, de-assert, then reconfigure csreg for the next target.

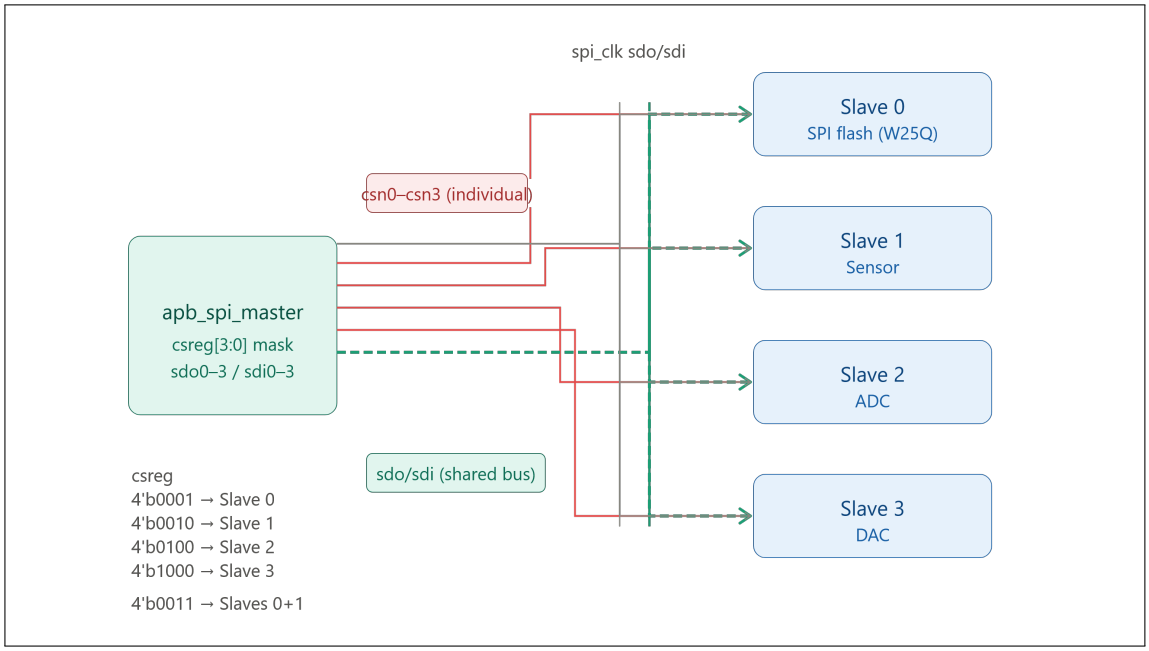


Figure 1: CSN lines (red) are independent. Data lines (green dashed) and spi_clk (gray) are shared by all slaves.

The key contrast the diagram above shows:

- **CSN lines (red):** One dedicated wire per slave. Each slave gets its own private chip select line running from the master directly to it. No sharing. `csreg` is the bitmask that decides which of these four wires gets pulled LOW during a transaction. Writing `4'b0001` asserts only the Slave 0 line. Writing `4'b0011` asserts both Slave 0 and Slave 1 simultaneously.
- **Data lines (green dashed) and `spi_clk` (gray):** Shared by all slaves. There is one physical wire for each of `sdo0/1/2/3` and `sdi0/1/2/3` running as a common bus that every slave connects to. The same bits travel down the same wire to every slave on the bus. The CSN lines are the only mechanism separating one slave's transaction from another's—a slave ignores all data when its own CSN is HIGH.

This is why multi-select (`csreg=4'b0011`) is a broadcast: the shared data lines deliver identical data to both Slave 0 and Slave 1 simultaneously, and both respond on `sdi1` at the same time—causing a bus conflict on the return path.

Operating Modes and Pin Utilization

Furthermore, the IP supports multiple transfer modes to scale data throughput. Depending on the configuration written to `REG_STATUS` (such as `spi_qrd` / `spi_qwr`), the hardware dynamically alters its pin utilization between Standard (Single) and Quad (QPI) modes. The RTL directly proves how the 4 data lines are utilized during both transmission and reception:

Listing 2: Operating Modes, TX and RX Shift Register Logic

```
// spi_master_controller.sv
`define SPI_STD     2'b00
`define SPI_QUAD_TX 2'b01
`define SPI_QUAD_RX 2'b10

// spi_master_apb_if.sv
                    `REG_STATUS:
                    begin
                        spi_rd    <= PWDATA[0];
                        spi_wr    <= PWDATA[1];
                        spi_qrd   <= PWDATA[2];
                        spi_qwr   <= PWDATA[3];
                        spi_swrst <= PWDATA[4];
                        spi_csreg <= PWDATA[11:8];
                    end

// spi_master_tx.sv
      assign sdo0 = (en_quad_in) ? data_int[28] : data_int[31];
      assign sdo1 = (en_quad_in) ? data_int[29] : 1'b0;
      assign sdo2 = (en_quad_in) ? data_int[30] : 1'b0;
      assign sdo3 = (en_quad_in) ? data_int[31] : 1'b0;

// spi_master_rx.sv
          if (en_quad_in)
            data_int_next = {data_int[27:0],sdi3,sdi2,sdi1,sdi0};
          else
            data_int_next = {data_int[30:0],sdi1};
```

The PREADY Wait Mechanism

The core synchronization relies on the APB `PREADY` signal as an implicit hardware wait-state. When the FSM attempts to read the SPI `RX_FIFO` immediately after triggering a read, the FIFO is initially empty because the autonomous SPI flash controller has not yet completed shifting the first 32-bit word.

Consequently, the APB interface dynamically drives `PREADY` low. **The Boot Controller safely stalls (waits) here**. On the exact clock cycle that the SPI controller reaches `spi_eot = 1` (End-Of-Transfer) and pushes the shifted word into the `RX_FIFO`, `PREADY` automatically asserts. This allows the FSM to wake up, instantly capture the word, and fire it over AXI-Lite.

Detailed FSM States

The Finite State Machine representing the Boot Controller processes exactly one 32-bit instruction per iteration, looping back to reconfigure the SPI master for the next address until the full firmware is loaded. **Note the explicit wait states in both the APB and AXI domains.**

Table 1: Boot Controller FSM States and Actions

State Name	Action Description	Transition Condition
IDLE	Wait for external reset.	Single-cycle reset pulse → LATCH_RESET
LATCH_RESET	Latch the reset event. Assert `core_halt = 1` to freeze the CPU. Initialize addresses.	Unconditional → CFG_SPI_MASTER
CFG_SPI_MASTER	APB writes to configure the SPI master for a single 32-bit read at `current_flash_addr`.	APB Writes Complete → TRIGGER_READ
TRIGGER_READ	APB Write: Start the SPI hardware transfer.	APB Write Complete → READ_FIFO
READ_FIFO	APB Read: REG_RXFIFO. **FSM Waits (stalls) here**. It sleeps while PREADY=0 until the SPI IP fills the FIFO.	PREADY == 1 → AXI_WRITE
AXI_WRITE	AXI-Lite Write: Send captured word to target memory (I-SRAM/D-SRAM). **FSM Waits here** for crossbar ready.	BVALID & BREADY → CHECK_DONE
CHECK_DONE	Increment `flash_addr`, memory address, and word counter.	total < MAX → CFG_SPI_MASTER total == MAX → RELEASE_CORE
RELEASE_CORE	De-assert `core_halt = 0`. Handover bus to CPU.	Unconditional → DONE
DONE	Terminal state. CPU begins execution.	None (Wait for next hard reset)

Hardware Transfer Sequence

Configuration for Continuous Streaming

To achieve the continuous DMA stream, the Boot Controller FSM must execute a precise sequence of APB writes to configure the IP before pulling the trigger.

The critical component is the SPI Length register (REG_SPILEN). Depending on whether the bootloader chooses to segment the transfer into FIFO-depth safe batches (e.g., 10 words) or execute a single massive continuous read, this configuration varies.

Listing 3: Boot Controller FSM Configuration Sequence

```
// FSM State: CFG_SPI_MASTER

// 1. Set SPI clock divider
//    SPI clock period = (clk_div + 1) x 2 HCLK cycles
//    clk_div=4 -> 10 HCLK cycles per SPI clock = 10ns per bit at 100MHz
apb_write(REG_CLKDIV, 32'h0000_0004);
```

```
// 2. Set Flash Read Command
//    Standard SPI flash slow-read opcode = 0x03 (8 bits)
apb_write(REG_SPICMD, 32'h0000_0003);

// 3. Set Flash Start Address
//    Read from address 0x000000 (start of flash)
apb_write(REG_SPIADR, 32'h0000_0000);

// 4. Set Transfer Lengths
//    Layout: PWDATA = {data_len[15:0], 2'b00, addr_len[5:0], 2'b00, cmd_len[5:0]}
//    cmd_len  = 6'd8     -> 8-bit opcode  (0x03)
//    addr_len = 6'd24    -> 24-bit address
//    data_len = 16'd320 -> 320 bits = 10 words (one FIFO-depth batch)
//    NOTE: FIFO depth = 10 words. Send max 10 words per transaction.
//          Loop 102 times (102 x 10 = 1020, then final batch of 4) for 1024 total.
//          OR use data_len = 16'd32768 for a single 1024-word burst but
//          the bootloader MUST drain mid-burst (see TC5 stall behaviour).
apb_write(REG_SPILEN, {16'd320, 2'b00, 6'd24, 2'b00, 6'd8});

// FSM State: TRIGGER_READ
// Layout: PWDATA = {20'h0, csreg[3:0], swrst, qwr, qrd, wr, rd}
// csreg[3:0] = 4'b0001 -> CS0 asserted (slave 0 selected)
// rd = 1               -> standard single-line read
// All other bits = 0
apb_write(REG_STATUS, 32'h0000_0101);
//                              ^^^^
//                              ||++-- [3:0]  = 0001 = rd=1, wr=0, qrd=0, qwr=0, swrst=0
//                              ++---- [11:8] = 0001 = csreg -> CS0 only selected
```

RTL Hardcoding of 32-bit Words

While the total transfer length may be extensive, the internal RTL logic of the `apb_spi_master` IP is strictly hardcoded to segment any incoming stream into native 32-bit chunks. Specifically, the internal receive module (`spi_master_rx.sv`) operates an active bit-counter:

```
assign reg_done = (!en_quad_in && (counter[4:0] == 5'b11111));
```

This logic asserts the `reg_done` flag precisely every time 32 bits are captured. This guarantees that the stream is automatically, hardware-sliced into exactly 32-bit words, pushed sequentially into the `RX_FIFO` perfectly formatted for AXI-Lite.

Hardware Transfer Sequence

Configuration for Continuous Streaming

To achieve the continuous DMA stream, the Boot Controller FSM must execute a precise sequence of APB writes to configure the IP before pulling the trigger.

The critical component is the SPI Length register (`REG_SPILEN`). Depending on whether the bootloader chooses to segment the transfer into FIFO-depth safe batches (e.g., 10 words) or execute a single massive continuous read, this configuration varies.

Listing 4: Boot Controller FSM Configuration Sequence

```
// FSM State: CFG_SPI_MASTER

// 1. Set SPI clock divider
//    SPI clock period = (clk_div + 1) x 2 HCLK cycles
//    clk_div=4 -> 10 HCLK cycles per SPI clock = 10ns per bit at 100MHz
apb_write(REG_CLKDIV, 32'h0000_0004);

// 2. Set Flash Read Command
//    Standard SPI flash slow-read opcode = 0x03 (8 bits)
apb_write(REG_SPICMD, 32'h0000_0003);
```

```
// 3. Set Flash Start Address
//    Read from address 0x000000 (start of flash)
apb_write(REG_SPIADR, 32'h0000_0000);

// 4. Set Transfer Lengths
//    Layout: PWDATA = {data_len[15:0], 2'b00, addr_len[5:0], 2'b00, cmd_len[5:0]}
//    cmd_len  = 6'd8    -> 8-bit opcode  (0x03)
//    addr_len = 6'd24   -> 24-bit address
//    data_len = 16'd320 -> 320 bits = 10 words (one FIFO-depth batch)
//    NOTE: FIFO depth = 10 words. Send max 10 words per transaction.
//          Loop 102 times (102 x 10 = 1020, then final batch of 4) for 1024 total.
//          OR use data_len = 16'd32768 for a single 1024-word burst but
//          the bootloader MUST drain mid-burst (see TC5 stall behaviour).
apb_write(REG_SPILEN, {16'd320, 2'b00, 6'd24, 2'b00, 6'd8});

// FSM State: TRIGGER_READ
// Layout: PWDATA = {20'h0, csreg[3:0], swrst, qwr, qrd, wr, rd}
// csreg[3:0] = 4'b0001 -> CS0 asserted (slave 0 selected)
// rd = 1               -> standard single-line read
// All other bits = 0
apb_write(REG_STATUS, 32'h0000_0101);
//                              ^^^^
//                              ||++-- [3:0]  = 0001 = rd=1, wr=0, qrd=0, qwr=0, swrst=0
//                              ++---- [11:8] = 0001 = csreg -> CS0 only selected
```

RTL Hardcoding of 32-bit Words

While the total transfer length may be extensive, the internal RTL logic of the `apb_spi_master` IP is strictly hardcoded to segment any incoming stream into native 32-bit chunks. Specifically, the internal receive module (`spi_master_rx.sv`) operates an active bit-counter:

```
assign reg_done = (!en_quad_in && (counter[4:0] == 5'b11111));
```

This logic asserts the `reg_done` flag precisely every time 32 bits are captured. This guarantees that the stream is automatically, hardware-sliced into exactly 32-bit words, pushed sequentially into the `RX_FIFO` perfectly formatted for AXI-Lite.

Decoupled Hardware Synchronization

The Boot flow relies on two distinct FSMs executing simultaneously:

- **SPI Master IP:** Once triggered by the Boot Controller, the SPI IP transitions out of its own wait state, takes control of the physical SPI pins, and autonomously shifts data bit-by-bit.
- **Boot Controller:** After pulling the trigger, the Boot Controller immediately enters a stalled wait state on the APB bus. It sleeps passively. The instant the SPI Master IP pushes a completed word into the RX_FIFO, the Boot Controller is physically unblocked, drains the FIFO, writes to AXI, and re-triggers the SPI Master IP for the next word.

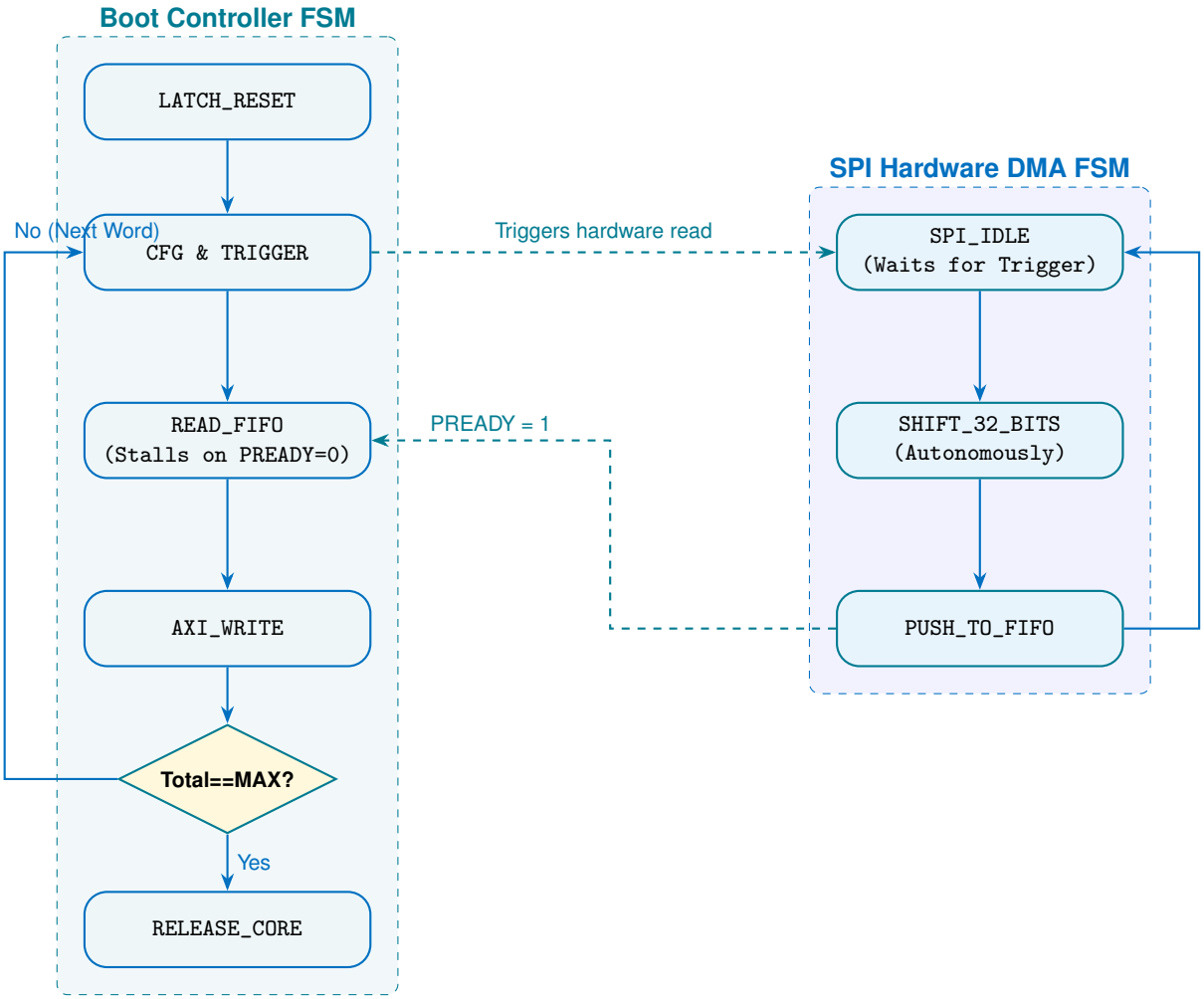


Figure 2: Parallel FSMs: Decoupled Hardware Synchronization

Hardware Timing Diagram (Waveform)

The waveform below explicitly details the timing relationship across the independent buses. It proves how the Boot Controller's APB read transaction is safely suspended, absorbing the entirety of the slow SPI transaction time, and immediately responding the cycle the data arrives.

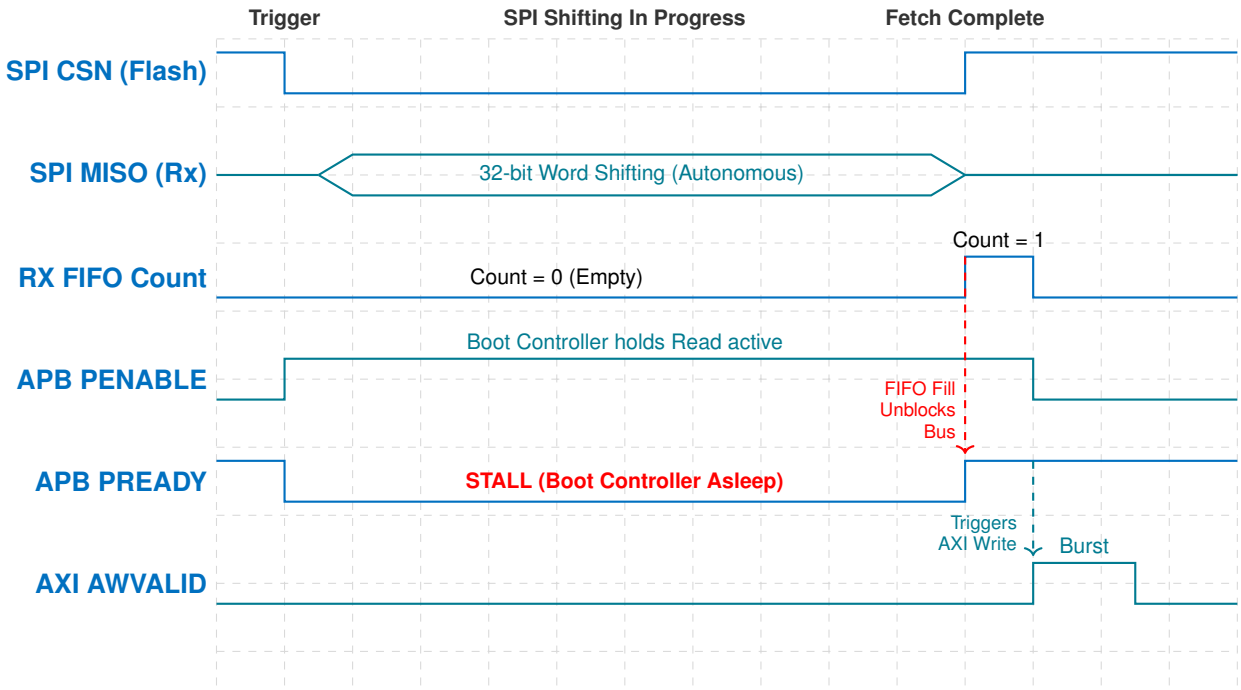


Figure 3: Cycle-Accurate Hardware Timing Sequence

Architectural Clarification: SPI Master vs. SPI Slave IPs

ASIC tape-outs are massive, high-stakes undertakings. It is incredibly common during the architectural phase to encounter confusion surrounding IP naming conventions. This section plainly clarifies the distinction between SPI Master and SPI Slave IPs.

The core distinction is **not** overall capability, but strictly **bus mastership direction**—specifically, which side of the interface generates the serial clock, and which side actively issues memory transactions.

Purpose of Each IP

`apb_spi_master`**:** This IP represents the SoC reaching outward to talk to an external passive chip. It generates the SPI clock and chip-select itself, and is controlled internally via APB writes from the RISC-V core or a boot controller. It has no intelligence of its own—every transaction it performs was explicitly commanded by software writing to its registers.

`axi_spi_slave`**:** This IP represents an external active device reaching inward into the SoC. It does not generate anything on the SPI side—it waits to be clocked by an outside host. But once it decodes a command from that external host, it becomes fully autonomous on the AXI side: it issues real AXI master transactions into the SoC's memory map with zero involvement from the RISC-V core. This exists for scenarios like an external tester, FPGA, or debug host needing direct memory access into the chip without going through the CPU at all.

These two IPs solve opposite problems. Neither is a substitute for the other.

Physical Connections and Tapeout Implications

Connecting the `apb_spi_master`:

Listing 5: SPI Master Port Signatures (`apb_spi_master.sv`)

```
    output logic                [1:0] events_o,

    output logic                      spi_clk,
    output logic                      spi_csn0,
```

```
    output logic                        spi_csn1,
    output logic                        spi_csn2,
    output logic                        spi_csn3,
    output logic                 [1:0]  spi_mode,
    output logic                        spi_sdo0,
    output logic                        spi_sdo1,
    output logic                        spi_sdo2,
    output logic                        spi_sdo3,
    input  logic                        spi_sdi0,
    input  logic                        spi_sdi1,
    input  logic                        spi_sdi2,
    input  logic                        spi_sdi3
```

At the pad ring, these are straightforward: `spi_clk`, `spi_csn0`, and `spi_sdo0` go to output pads, and `spi_sdi1` goes to an input pad. There is no special timing closure required beyond standard output pad timing, as the SoC is the timing reference.

Connecting the `axi_spi_slave`:

Listing 6: SPI Slave Port Signatures (`axi_spi_slave.sv`)

```
    input  logic                        test_mode,
    input  logic                        spi_sclk,
    input  logic                        spi_cs,
    output logic  [1:0]                 spi_mode,
    input  logic                        spi_sdi0,
    input  logic                        spi_sdi1,
    input  logic                        spi_sdi2,
    input  logic                        spi_sdi3,
    output logic                        spi_sdo0,
    output logic                        spi_sdo1,
    output logic                        spi_sdo2,
    output logic                        spi_sdo3,
```

This presents a critical tapeout difference. Confirmed by the RTL, `axi_spi_slave` has only **one** `spi_cs` input, unlike the master's four outputs. More importantly, `spi_sclk` is an asynchronous external input pad with no relationship to the chip's internal clock domains.

You need a robust Clock Domain Crossing (CDC) strategy. The RTL handles this internally via `spi_slave_dc_fifo.sv` because the SPI-side logic runs entirely on the external `spi_sclk` domain, while the AXI side runs on the internal `axi_aclk`. For tapeout, you must instantiate proper input pad cells with sufficient setup/hold margin for an unsynchronized external clock. You cannot apply normal STA clock constraints to `spi_sclk` like an internal PLL-derived clock—it must be constrained as a fully asynchronous/external clock domain in the SDC timing constraints.

Connecting an Actual SPI Flash Chip

Connecting a flash chip uses the `apb_spi_master` only. The `axi_spi_slave` is physically incompatible with a flash chip.

Pin mapping to a standard SPI NOR flash (e.g., W25Q series):

apb_spi_master pin	Flash chip pin	Direction
`spi_clk`	CLK	SoC → Flash
`spi_csn0`	CS#	SoC → Flash
`spi_sdo0`	DI (MOSI)	SoC → Flash
`spi_sdi1`	DO (MISO)	Flash → SoC
`spi_sdo1/2/3`, `spi_sdi0/2/3`	IO1/IO2/IO3 (quad)	Bidirectional (Quad mode only)

If you want to connect multiple SPI peripherals on the same bus (e.g., flash + a sensor), each gets its own dedicated chip-select:

- `spi_clk` → shared by all devices
- `spi_sdo0` → shared MOSI line to all devices

- `spi_sdi1` → shared MISO line from all devices
- `spi_csn0` → Flash chip's CS#
- `spi_csn1` → Second device's CS#
- `spi_csn2` → Third device's CS#
- `spi_csn3` → Fourth device's CS#

By setting `csreg` (a bitmask), you assert exactly one CS line per transaction. The data lines are shared, but only the addressed device should be driving `spi_sdi1` at any time. Every non-selected device **must tristate** its output when its own CS is high. This is standard SPI bus practice, but it is critical to confirm each datasheet actually tristates correctly before tapeout. A misbehaving slave driving the shared MISO line while deselected will corrupt every other transaction on the bus.

Hardware SPI Slave Chip vs. `axi_spi_slave` IP

This is the most critical distinction for tapeout planning. A real hardware SPI slave (like a sensor, an ADC, or a flash chip in slave-addressable mode) is a dumb responder. It:

- Has no clock generator of its own.
- Has no command parser issuing transactions into anything.
- Simply shifts bits in/out synchronized to whatever clock the master gives it.
- Cannot initiate communication—ever.

The `axi_spi_slave` IP is **not** this. It is an SPI slave only on its external pins—meaning it waits to be clocked by an external master. But internally, it is an active AXI bus master. The moment it decodes a valid command byte (`0x02` for write-memory, `0x0B` for read-memory, confirmed in `spi_slave_cmd_parser.sv`), it autonomously drives `axi_master_aw_valid`, `axi_master_w_data`, etc., directly into the SoC's address space—completely bypassing the RISC-V core.

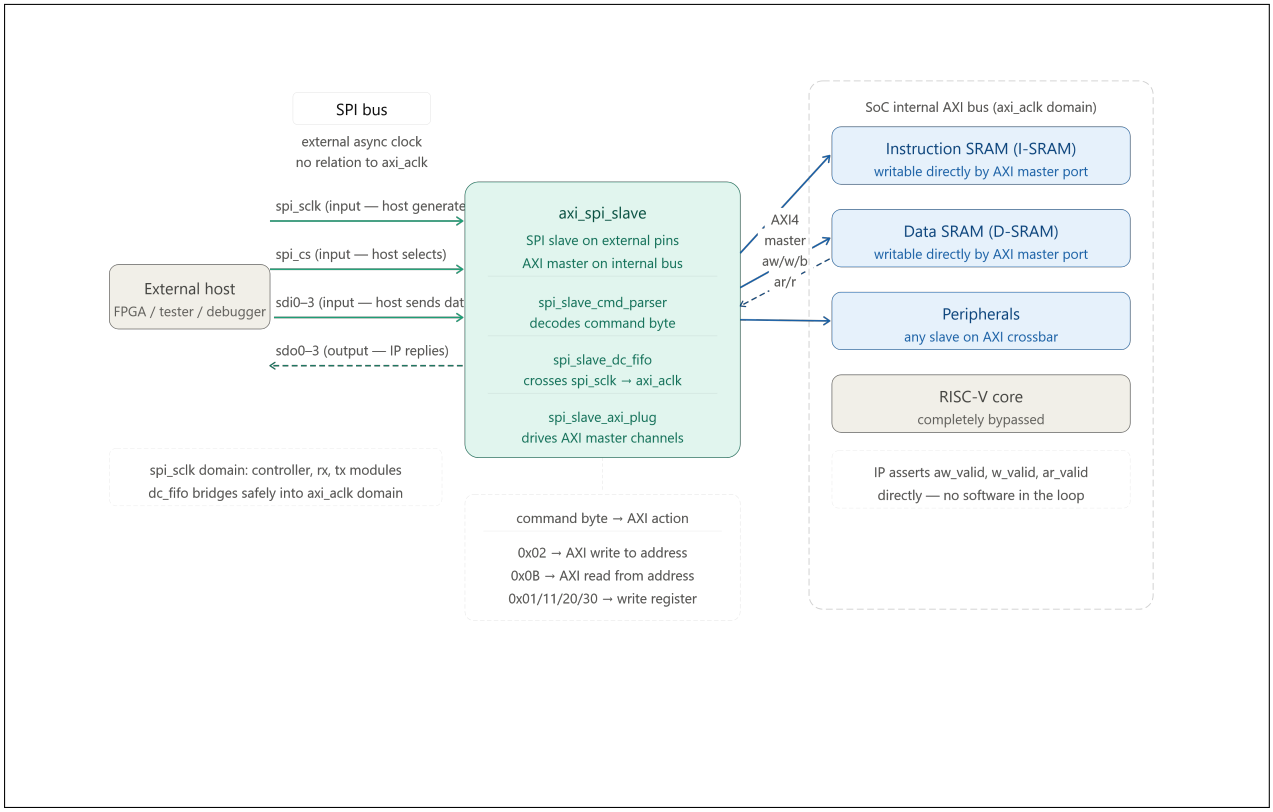


Figure 4: The `axi_spi_slave` acting as an active AXI master into the SoC.

Listing 7: SPI Slave Command Parser Decode (spi_slave_cmd_parser.sv)

```
      8'h2: //write mem
      begin
        get_addr       = 1;
        get_mode       = 0;
        get_data       = 1;
        send_data      = 0;
        enable_cont    = 1'b1;
        enable_regs    = 1'b0;
        error          = 1'b0;
        wait_dummy     = 0;
        reg_sel        = 2'b00;
      end
      8'hB: //read mem
      begin
        get_addr       = 1;
        get_mode       = 0;
        get_data       = 0;
        send_data      = 1;
        enable_cont    = 1'b1;
        enable_regs    = 1'b0;
        error          = 1'b0;
        wait_dummy     = 1;
        reg_sel        = 2'b00;
      end
```

The "SPI Slave" in the IP's name describes its behavior on the physical SPI pins only. A real hardware SPI slave chip you solder onto a board is passive end-to-end. axi_spi_slave is passive externally but a fully privileged bus master internally.

Important

Tapeout Security Warning: If you instantiate axi_spi_slave in your design, you give whatever external device connects to those pins direct, unrestricted write access to your entire AXI memory map. There are no permission checks visible in this RTL. While appropriate for a trusted debug/test interface during bring-up, if those pins are exposed on a production board without access control, it is a genuine security surface. Anyone with physical access can write arbitrary data to arbitrary addresses, potentially overwriting boot code or security-critical registers.